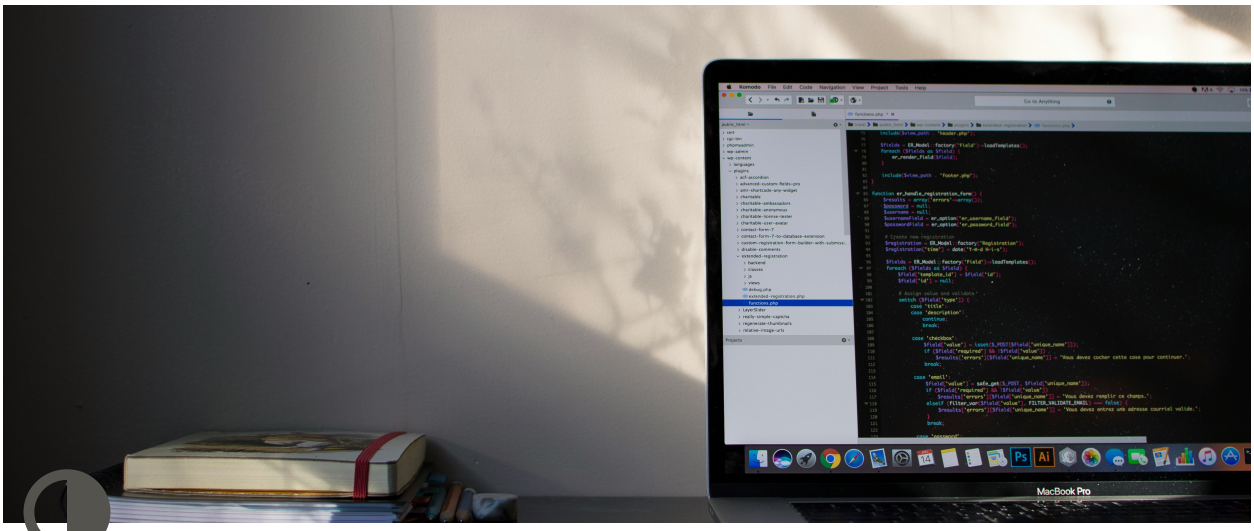


Java

Status	Not started

Java, JDK, JRE, and JVM

```
Java Code --> Compiler (javac) --> Byte Code --> JVM --> Output
```

1. JDK

- JDK means **Java Development Kit**.
- It is the complete software development kit.
- It includes the JRE and development tools.
- It is used by developers to build programs.
- It contains tools such as `javac`, `java`, `jdb`, `jar`, and `javadoc`.

2. JRE

- JRE provides an environment to run a Java application.

- It contains the JVM and standard class libraries.
- It does not contain development tools.
- It is used by end users.

3. JVM

- JVM is a core part of the JRE.
- It loads, verifies, and executes bytecode.
- It provides platform independence.

Basic syntax

```
class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Identifiers, Variables, Data Types, Literals, and Casting

Identifiers and naming rules

- An identifier is a name used for a class, variable, method, and so on.
- It can contain letters (`A-Z`, `a-z`), digits (`0-9`), underscore (`_`), and dollar sign (`$`).
- It must start with a letter, `_`, or `$`; it must not start with a digit.
- Special characters are not allowed.
- Identifiers are case-sensitive.

Variables

- A variable is a named memory location used to store data.

- Variables are case-sensitive.

Types of variables

1.Local variable

- Declared inside a method or block.
- Has no default value.

2.Instance variable

- Declared inside a class but outside methods.
- Has a default value.

3.Static variable

- Declared using the `static` keyword.
- Belongs to the class.

```
class Demo {
    static int count = 0; // static variable
    int id;               // instance variable

    void show() {
        int x = 10;       // local variable
        System.out.println(x);
    }
}
```

Data types

Data types define what type of data a variable can store.

Primitive / built-in data types

- Integer: `byte`, `short`, `int`, `long`
- Floating point: `float`, `double`

- Character: `char`
- Boolean: `boolean`

Type	Size	Range / precision
`byte`	1 byte	-128 to 127
`short`	2 bytes	-32,768 to 32,767
`int`	4 bytes	-2^31 to 2^31 - 1
`long`	8 bytes	-2^63 to 2^63 - 1
`float`	4 bytes	about 6-7 decimal digits
`double`	8 bytes	about 15 decimal digits
`char`	2 bytes	Unicode character

Non-primitive / reference data types

- String
- Array
- Class
- Interface
- Enum

Literals

A literal is a fixed value written directly in Java source code.

Types of literals

- Integer
- Floating point
- Character
- String
- Boolean

- null

Integer literals can be written in:

- Decimal (0 to 9)
- Octal (0 to 8)
- Hexadecimal (0 to 9 / A to F)

Type casting

1. Implicit casting

- The compiler is responsible for performing it.
- A smaller data-type value is assigned to a bigger data-type variable.
- It is also known as widening or upcasting.
- There is no loss of information.

2. Explicit casting

- The programmer is responsible for performing it.
- A bigger data-type value is assigned to a smaller data-type variable.
- It is also known as narrowing or downcasting.
- There is a chance of losing information.

```
int number = 10;
double largerValue = number; // implicit casting

double price = 10.75;
int smallerValue = (int) price; // explicit casting
```

```
byte -> short -> int -> long -> float -> double
char -> int -> long -> float -> double
```

```
Implicit: left to right
Explicit: right to left
```

Operators, Conditions, and Loops

Arithmetic operators

Arithmetic operators are used to perform mathematical calculations.

1. Addition: `+`
2. Subtraction: `-`
3. Multiplication: `*`
4. Division: `/`
5. Modulus: `%`
6. Increment and decrement: `++`, `--`

```java
int number = 10;
int first = ++number; // increment, then fetch the value
int second = number++; // fetch the value, then increment
```

Relational operators

Relational operators are used to compare two values.

- `==` equal to
- `!=` not equal to
- `>` greater than
- `<` less than
- `>=` greater than or equal to
- `<=` less than or equal to

Logical operators

Logical operators are used to combine conditions.

- `&&` - AND
- `||` - OR
- `!` - NOT; `false` becomes `true`, and `true` becomes `false`

Conditional statements

Conditional statements are used to make decisions. They check whether a condition is true or false and then execute code.

`if` statement

Used when a condition is true.

```
if (condition) {
    // code
}
```

`if-else` statement

```
if (condition) {
    // true block
} else {
    // false block
}
```

`else-if` ladder

Used when there are multiple conditions.

```
if (condition1) {
    // code
} else if (condition2) {
    // code
} else if (condition3) {
    // code
} else {
    // code
}
```

Ternary operator

- Also called the conditional operator.
- It is the short form of an `if-else` statement.

```
result = condition ? expression1 : expression2;
```

`switch` statement

Useful when one expression is compared against multiple possible values/cases.

```
switch (expression) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code
}
```

Loop control instructions

Loops repeat some parts of a program.

1. `for` loop

Executes a block of code repeatedly as long as a given condition is satisfied.

```
for (initialization; condition; update) {
    // statement
}
```

2. `while` loop

Executes code as long as the condition is true. The condition is checked before executing the loop body.

```
while (condition) {
    // statements
}
```

3. `do-while` loop

- The loop executes at least once.
- It executes code first and checks the condition afterward.

```
do {
    // statement
} while (condition);
```

Enhanced `for` loop / `for-each`

Used to traverse an array without using an index.

```java
int[] array = {10, 20, 30};

for (int x : array) {
    System.out.println(x);
}
```

Class, Object, Methods, Stack, and Heap

Class vs Object

Class

- A class is a blueprint, template, or logical data type used to define properties and behaviors.
- A class does not occupy memory when it is defined.
- It is a purely logical entity.
- A class is defined only once.

Object

- An object is a physical instance of a class created in memory.
- Memory is allocated on the heap as soon as it is created.
- It is a physical entity with state, behavior, and unique identity.
- Multiple objects can be created from a single class.

```java
class Student {
    String name;
    int age;

    Student(String name, int age) {
```

```
        this.name = name;
        this.age = age;
    }

    void study() {
        System.out.println(name + " is studying");
    }

    public static void main(String[] args) {
        Student student1 = new Student("Aarav", 20);
        student1.study();
    }
}
```

Method in Java

A method is a block of code designed to perform a specific task. It runs only when it is called.

Anatomy of a Method

```
modifier returnType methodName(parameters) {
    // code (method body)
    return value;
}
```

- **Modifier:** defines who can use the method, for example `public`, `private`, or `static`.
- **Return type:** the data type of the value the method sends back, for example `int` or `String`.
- **Method name:** a unique name used to call the method; it is written in camelCase.
- **Parameters:** variables passed into the method as input.
- **Method body:** the actual code inside curly braces `{}`.

Method types

- **Instance method:** requires an object created using `new` before the method can be called.
- **Static method:** belongs to the class. Call it directly with `ClassName.methodName()`.

Method overloading

Method overloading means having multiple methods with the same name but different parameters.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

In method overloading, change one of the following:

- **Number of parameters**

```
add(int a, int b);
add(int a, int b, int c);
```

- **Type of parameters**

```
add(int a, int b);
add(double a, double b);
```

- **Order of different parameters**

```
display(int a, String b);
display(String a, int b);
```

Stack and heap in Java

1. Stack

- Stores method-execution information, stack frames, and local variables.
- Each method call creates a stack frame.
- When a method finishes, its stack frame is removed.
- Works in LIFO order.
- Stack memory is smaller and faster than heap memory.
- `StackOverflowError` can occur when stack memory is exhausted.

2. Heap

- Stores objects and arrays created using `new`.
- It is managed by garbage collection.
- Objects remain in the heap until they are no longer reachable.
- Heap memory is larger than stack memory.
- `OutOfMemoryError` can occur when the JVM cannot allocate more heap memory.

```
Student s1 = new Student("Aarav", 20);
int age = 20;
```

In this example, `s1` and `age` are local references/values in the stack frame; the `Student` object is in the heap.

Arrays in Java

1. Need for arrays

- An array stores multiple values of the same data type in one variable.
- Elements are accessed using an index.
- It is useful when the required number of elements is known.

2. Creating an array

```
int[] array;              // declaration
array = new int[5];       // creation

int[] values = {10, 20, 30, 40, 50};
```

- Array size is fixed after creation.

3. Multidimensional array

- A multidimensional array is an array containing another array.
- It is most commonly used as a 2D array.
- It is useful for tables and matrices.

```
int[][] array = {
    {1, 2, 3},
    {4, 5, 6}
};
```

4. Jagged array

A jagged array is a 2D array in which each row can have a different length.

```
int[][] array = {
    {1, 2},
    {3, 4, 5},
```

```
    {6}
};
```

Java supports jagged arrays because a 2D array is actually an array of arrays.

5. 3D array

A 3D array contains 2D arrays.

```
int[][][] array = new int[2][3][4];
```

- `2` - layers
- `3` - rows
- `4` - columns

6. Drawbacks of arrays

- Fixed size; an array cannot grow or shrink after creation.
- It stores elements of the same data type.
- Insertion/deletion in the middle can require shifting elements.
- Allocating more space than required can waste memory.

Strings, StringBuffer, and StringBuilder

1. String

- `String` is a class in Java used to represent a sequence of characters.
- Strings are objects, not primitive data types.
- Strings can be created in two common ways.

```
String s1 = "Hello";
String s2 = new String("Hello");
```

2. Immutable strings

Once a `String` object is created, its value cannot be changed.

```
String s = "Hello";
s = s + " World";
```

The original `"Hello"` object is not modified; a new `String` object is created.

Why strings are immutable

- Security
- Thread safety
- String-pool optimization
- They allow string objects to be safely used

3. Mutable strings

A mutable object can be modified after creation.

```
StringBuffer sb = new StringBuffer("Hello");
sb.append(" World");
System.out.println(sb);
```

4. StringBuffer

- A class used to create mutable strings.
- Its content can change without creating a new object for every modification.
- Its methods are synchronized, so `StringBuffer` is thread-safe.
- It is slower than `StringBuilder` because of synchronization.

5. StringBuilder

- A Java class used to create mutable sequences of characters.
- It is used when strings need to be modified frequently.

- It is not synchronized and is generally faster than `StringBuffer` in single-threaded code.

Important methods

```
StringBuilder sb = new StringBuilder("Hello");

sb.append(" Java");  // add at the end
sb.insert(5, ",");    // insert
sb.delete(0, 5);       // delete
sb.reverse();          // reverse
int length = sb.length(); //length
String text = sb.toString(); // convert to String
```

---

static, Encapsulation, this, and Constructors

static

- `static` means that a member belongs to the class, not to an individual object.
- Static members can be accessed using the class name.
- Only one shared copy of a static variable exists per class.

1. Static variable

```
class Student {
    static String college = "RGPV";
    String name;
}

System.out.println(Student.college);
```

2. Static method

```
class Calculator {
    static int add(int a, int b) {
        return a + b;
    }
}

int result = Calculator.add(10, 20);
```

- A static method belongs to the class.
- It can be called using the class name.
- A static method can directly access static members.
- It cannot directly access non-static variables because they belong to objects.
- `main()` is static because the JVM needs to call it without creating an object.

3. Static block

A static block is a block declared with `static`.

```
class Demo {
    static {
        System.out.println("Static block executed");
    }
}
```

- Used for static initialization.
- Executes automatically when the class is loaded.
- Generally runs once per class loading.
- Multiple static blocks execute in the order in which they appear.

```
class Demo {
    static { System.out.println("Block 1"); }
    static { System.out.println("Block 2"); }
```

```
    public static void main(String[] args) {
        System.out.println("Main");
    }
}
```

Encapsulation

Encapsulation means wrapping data and methods together in a class and controlling access to the data.

- Variables are usually `private`.
- Access happens through public getters and setters.

```
class Student {
    private int age;

    public void setAge(int age) {
        this.age = age;
    }

    public int getAge() {
        return age;
    }
}
```

Why Encapsulation?

- Protect data from direct access.
- Provide controlled access.
- Improve security and maintainability.

Getters and setters

Getter - used to get/read a value.

```
public int getAge() {
    return age;
}
```

Setter - used to set/change a value.

```
public void setAge(int age) {
    this.age = age;
}
```

this keyword

- `this` refers to the current object.

```
class Student {
    int age;

    Student(int age) {
        this.age = age; // this.age: instance variable; age:
constructor parameter
    }
}
```

Common uses

- Distinguish an instance variable from a parameter.
- Call another constructor using `this(...)`.
- Call a current-object method.
- Pass the current object as an argument.

`this` cannot be used in a static context because static members do not belong to a particular object.

Constructors

A constructor is a special member used to initialize an object.

```
class Student {
    int age;

    Student() {
        age = 20;
    }
}

Student s = new Student();
```

Important properties

- Constructor name equals the class name.
- It has no return type, not even `void`.
- It is automatically called when an object is created.
- It is mainly used for object initialization.
- It can be overloaded.

Default vs Parameterized Constructor

Default / no-argument constructor - a constructor with no parameters. It initializes with default or fixed values.

```
class Student {
    Student() {
        System.out.println("Student created");
    }
}
```

Parameterized constructor - a constructor that accepts parameters. It initializes with supplied values.

```
class Student {
    int age;

    Student(int age) {
        this.age = age;
    }
}
```

Naming, Inheritance, Polymorphism, and Access Control

Naming conventions

- Class: PascalCase - `Student`, `BankAccount`
- Variable: camelCase - `studentName`, `totalMarks`
- Method: camelCase - `getName()`, `printDetails()`
- Constant: UPPER_SNAKE_CASE - `MAX_VALUE`
- Package: lowercase - `com.company.project`

Anonymous object

An anonymous object is created without storing its reference in a variable.

```
new Student().display();
```

Use it when an object is needed only once and will not be reused.

Inheritance in Java

Need for inheritance

- Code reusability
- Method overriding
- Runtime polymorphism
- Easy maintenance
- Represents an **is-a** relationship

What is inheritance?

Inheritance is a mechanism in which a child class acquires the accessible properties and methods of a parent class.

```
class Child extends Parent {
}
```

Base class -> derived class.

Single inheritance

One derived class inherits from one base class.

```
Student -> CollegeStudent
```

Multilevel inheritance

Inheritance in a chain: one class inherits from another, and a third class inherits from the second.

```
Student
  -> CollegeStudent
       -> EngineeringStudent
```

Multiple inheritance

Multiple inheritance means one derived class inherits from more than one base class.

Java does not support multiple inheritance using classes because it can create ambiguity. Java supports it through interfaces.

```
class CollegeStudent implements Student, SportsStudent {
    // implementation
}
```

Polymorphism

Polymorphism means **many forms**.

- The same method name or reference can behave differently in different situations.
- One name can have many forms/behaviors.

Types of polymorphism

1. **Compile-time polymorphism** - achieved through method overloading.
2. **Runtime polymorphism** - achieved through method overriding.

Method overriding: a derived class provides its own implementation of a parent/base-class method. The method must have the same name and same parameters in both base and derived classes.

Why polymorphism is needed

- Flexibility
- Code reusability
- Loose coupling
- Support for method overloading and overriding
- Programs become easier to extend and maintain.

Access modifiers

Access modifiers define the visibility/accessibility of classes, variables, methods, and constructors.

1. `private`
2. `default`
3. `protected`
4. `public`

Private

Private members can be accessed only inside the same class.

```
class Student {
    private int marks;

    void display() {
        System.out.println(marks);
    }
}
```

Default

When no access modifier is written, it is default access. It can be accessed within the same package.

```
class Student {
    int marks;
}
```

Protected

A protected member can be accessed:

- Within the same class.

- Within the same package.
- In a subclass in another package.

```
class Student {
    protected int marks;
}
```

Public

A public member can be accessed from anywhere.

```
class Student {
    public String name;
}
```

this and super

`this` refers to the current object.

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Main uses of `this`:

- Access current-object variables.
- Call a current-object method.

- Call another constructor of the same class.

`super` refers to the immediate base/parent class. It is mainly used when inheritance is present.

Uses of `super`:

- Access a parent-class variable.
- Call a parent-class method.
- Call a parent-class constructor.

Packages

A package is a group of related classes, interfaces, and sub-packages.

It is used to:

- Organize Java code.
- Avoid naming conflicts.
- Provide access control.
- Make code easier to maintain.

```
package college;

public class Student {
    // code
}
```

Types of packages

1. Built-in packages, already provided by Java: `java.lang`, `java.util`, `java.io`.
2. User-defined packages, created by the programmer.

To use a package, use the `import` keyword.

```
import college.Student;
```

Why packages are used

- Organization: keep related classes together.
- Name conflicts: allow same-named classes in different packages.
- Access control: work with access modifiers.
- Reusability: classes can be imported and reused.

Dynamic method dispatch

- Also known as runtime polymorphism.
- It is the mechanism by which a call to an overridden method is resolved at runtime rather than compile time.

Why useful?

- Loose coupling and extensibility.
- Cleaner code.

final keyword

`final` means something cannot be changed further, depending on where it is used.

Final variable

Its value cannot be changed.

```
final int age = 20;
// age = 25; // error
```

Final method

A child class cannot override it.

```
class Student {
    final void study() {
```

```
        System.out.println("Studying");
    }
}

class EngineeringStudent extends Student {
    // void study() {} // error
}
```

Final class

A final class cannot be inherited.

```
final class Student {
}

// class EngineeringStudent extends Student {} // error
```

Object Methods, Wrappers, Abstract Classes, Inner Classes, and Interfaces

toString(), equals(), and hashCode()

`toString()` returns a string representation of an object.

```
class Student {
    int id = 101;
    String name = "Rahul";

    @Override
    public String toString() {
        return id + " " + name;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s);
    }
}
```

`equals()` is used to compare objects for logical equality.

```
Student s1 = new Student();
Student s2 = new Student();
System.out.println(s1.equals(s2));
```

`hashCode()` returns an integer hash value representing an object.

```
Student s = new Student();
System.out.println(s.hashCode());
```

If you override `equals()`, you should generally override `hashCode()` consistently.

Wrapper classes

Wrapper classes convert primitive data types into objects.

Primitive	Wrapper
`int`	`Integer`
`double`	`Double`
`char`	`Character`
`short`	`Short`
`long`	`Long`
`float`	`Float`

```java
int a = 10;
Integer b = a; // autoboxing

Integer boxed = 10;
int unboxed = boxed; // unboxing
```

Wrapper classes are needed because collections work with objects, not primitive types.

```java
ArrayList<Integer> numbers = new ArrayList<>();
// ArrayList<int> numbers = new ArrayList<>(); // error
```

abstract keyword

`abstract` is used to create incomplete classes or methods that are intended to be implemented by subclasses.

Abstract method - has no body.

```java
abstract class Student {
    abstract void study();
}
```

Abstract class can contain:

- Abstract methods
- Concrete methods
- Variables
- Constructors

```java
abstract class Student {
    abstract void study();

    void attendClass() {
        System.out.println("Attending class");
```

```
    }
}
```

You cannot create an object directly from an abstract class.

Inner class

A class defined inside another class is called an inner/nested class.

```
class College {
    class Student {
        void display() {
            System.out.println("Student");
        }
    }
}
```

Why use an inner class?

- Logical grouping.
- Better encapsulation.
- The inner class can access members of the outer class.

Types of inner classes

1. Member inner class - non-static class inside another class.
2. Static nested class - static class inside another class.
3. Local inner class - class declared inside a method/block.
4. Anonymous inner class - class without a name, created and used immediately.

Interface

An interface is a blueprint/contract that specifies what a class must do without necessarily specifying how it does it.

```
interface Student {
    void study();
}
```

Why use an interface?

1. **Abstraction:** hides implementation details and exposes only required behavior.
2. **Multiple inheritance:** Java does not allow `class C extends A, B`, but a class can implement multiple interfaces.

```
class Student implements A, B {
}
```

1. **Loose coupling:** code can depend on an interface instead of a specific implementation.
2. **Standard/contract:** defines rules that implementing classes must follow.

Types of interfaces

Normal interface

Contains two or more abstract methods.

```
interface Student {
    void study();
    void attendClass();
}
```

Functional interface

Contains exactly one abstract method.

```
@FunctionalInterface
interface Student {
```

```
    void study();
}
```

It can be used with lambda expressions.

Marker interface

Contains no methods or fields that define behavior.

```
interface Student {
}
```

It provides metadata or indicates a special capability.

Enum

An enum (enumeration) is a special type used to define a fixed set of constant values.

```
enum Semester {
    FIRST,
    SECOND,
    THIRD,
    FOURTH
}

Semester semester = Semester.THIRD;
// Semester semester = "THIRD"; // error
```

- An enum is actually a special kind of class.
- Enum constants are predefined constant objects of the enum type.
- Use an enum when possible values are known and limited.

Important enum methods

- `values()` - returns all enum constants.

- `valueOf()` - converts a matching `String` to an enum constant.
- `ordinal()` - returns the position/index of a constant, starting at `0`.

Annotations and lambda expressions

An annotation provides metadata/information about Java code. It does not directly change program logic.

```
@Override
void study() {
    System.out.println("Studying");
}
```

Common annotations

- `@Override` - indicates a method overrides a parent method.
- `@Deprecated` - marks code as outdated.
- `@SuppressWarnings` - suppresses compiler warnings.
- `@FunctionalInterface` - ensures an interface has exactly one abstract method.

A lambda expression is a short way to write the implementation of a functional interface. Lambdas were introduced in Java 8.

```
Student s = () -> {
    System.out.println("Studying Java");
};
```

Lambda expression with return

```
interface Calculator {
    int add(int a, int b);
}
```

```
Calculator c1 = (a, b) -> {
    return a + b;
};

Calculator c2 = (a, b) -> a + b;

System.out.println(c1.add(10, 20));
```

When the lambda body is a single expression, braces and `return` are not needed.

Exceptions and Exception Handling

What is an exception?

An exception is an abnormal event that occurs during program execution and can interrupt the normal flow of a program.

Exceptions can occur at:

- Runtime
- Compile time
- Logical level

```
int a = 10;
int b = 0;
System.out.println(a / b); // ArithmeticException: division by zero is not allowed

int[] array = {10, 20, 30};
System.out.println(array[5]); // ArrayIndexOutOfBoundsException
```

Common exceptions

- `ArithmeticException`

- `NullPointerException`
- `ArrayIndexOutOfBoundsException`
- `NumberFormatException`
- `ClassCastException`
- `IOException`

Exception handling using `try-catch`

Use `try-catch` to handle exceptions so the program can respond instead of terminating abruptly.

```
try {
    // risky code
} catch (ExceptionType e) {
    // handling code
}
```

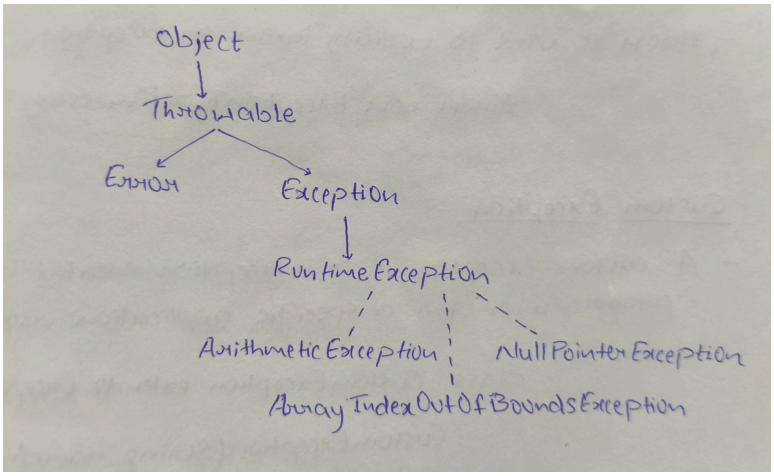
`try` with multiple `catch` blocks

A single `try` block can have multiple `catch` blocks.

```
try {
    int[] array = {10, 20};
    System.out.println(array[5]);
} catch (ArithmeticException e) {
    System.out.println("Arithmetic error");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid array index");
} catch (Exception e) {
    System.out.println("Some other exception");
}
```

Java checks catch blocks from top to bottom, so specific exceptions must come before general exceptions.

Exception hierarchy



There are two important branches under `Throwable`.

1. Error

- Serious problems generally not intended to be handled by normal application code.
- Examples: `OutOfMemoryError`, `StackOverflowError`.

2. Exception

- Problems that applications can often handle.
- Examples: `IOException`, `SQLException`, `ArithmeticException`, `NullPointerException`.

Checked exception - checked by the compiler. Examples: `IOException`, `SQLException`.

Unchecked exception - occurs at runtime. Examples: `ArithmeticException`, `NullPointerException`.

thorw, custom exceptions, and throws

`throw` is used to explicitly throw an exception.

```
throw new ExceptionType("message");
```

A `custom exception` is created by the programmer for a specific application requirement.

```
class CustomException extends Exception {
    CustomException(String message) {
        super(message);
    }
}
```

`throws` is used in a method declaration to say: “This method may throw this exception; the caller must handle or further declare it.” This is exception propagation.

```
void read() throws IOException {
    // code
}
```

```
readData()
  -> throws IOException
      -> caller handles it
```

User Input, BufferedReader, and Try-with-Resources

User input using `Scanner`

`Scanner` can directly read different data types.

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
```

- `Scanner` is a Java class.

- `System.in` represents the keyboard/input stream.
- `sc` is the Scanner object.

Common methods

```
int number = sc.nextInt();
double amount = sc.nextDouble();
String word = sc.next();
String line = sc.nextLine();
boolean answer = sc.nextBoolean();
```

User input using BufferReader

`BufferedReader` reads input as text/strings.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;

BufferedReader br = new BufferedReader(
    new InputStreamReader(System.in)
);

String s = br.readLine();
int n = Integer.parseInt(s);
```

To convert input text to an integer, use `Integer.parseInt(s)`.

Try-with-resources

Try-with-resources automatically closes resources after use.

```
try (BufferedReader br = new BufferedReader(
        new InputStreamReader(System.in))) {
    String data = br.readLine();
```

```
    System.out.println(data);
} catch (IOException e) {
    System.out.println("File error");
}
```

After the `try` block finishes, `br` is automatically closed.

Why use it?

- Automatically closes resources.
- Prevents resource leaks.
- Produces cleaner code.
- Handles exceptions properly.

Common resources

- `BufferedReader`
- `FileReader`
- `FileInputStream`
- `Connection`
- `Statement`
- `ResultSet`

Try-with-resources was introduced in Java 7.

Threads and Thread Safety

Threads

Instead of doing everything sequentially, multiple tasks can make progress concurrently.

```
class StudentThread extends Thread {
    @Override
```

```
    public void run() {
        System.out.println("Student is studying");
    }
}

StudentThread t = new StudentThread();
t.start();
```

- `start()` creates/schedules a new thread, which then executes `run()`.
- Calling `run()` directly is a normal method call on the current thread.

Multiple threads

Multiple threads can run concurrently.

```
class A extends Thread {
    @Override
    public void run() {
        System.out.println("A");
    }
}

class B extends Thread {
    @Override
    public void run() {
        System.out.println("B");
    }
}

class Main {
    public static void main(String[] args) {
        A t1 = new A();
        B t2 = new B();
        t1.start();
```

```
        t2.start();
    }
}
```

You can get a thread priority using:

```
t1.getPriority();
```

Higher priority does not guarantee the thread executes first; it is only a scheduling hint to the JVM/operating system.

`sleep()`

`Thread.sleep()` pauses the currently executing thread for a specified amount of time.

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    System.out.println("Thread interrupted");
}
```

`1000` milliseconds = `1` second.

`Runnable` vs `Thread`

There are two common ways to create a thread.

Extending `Thread`

```
class StudentTask extends Thread {
    @Override
    public void run() {
        System.out.println("Studying");
    }
}
```

```
StudentTask t = new StudentTask();
t.start();
```

Implementing `Runnable`

```
class StudentTask implements Runnable {
    @Override
    public void run() {
        System.out.println("Studying");
    }
}

StudentTask task = new StudentTask();
Thread t = new Thread(task);
t.start();
```

Race condition

A race condition occurs when multiple threads access and modify the same shared data concurrently and the final result depends on the timing of operations.

```
class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}
```

Suppose two threads each execute `counter.increment()` 1,000 times. The expected result is 2,000, but `count++` is not a single operation:

```
read count
add 1
write count
```

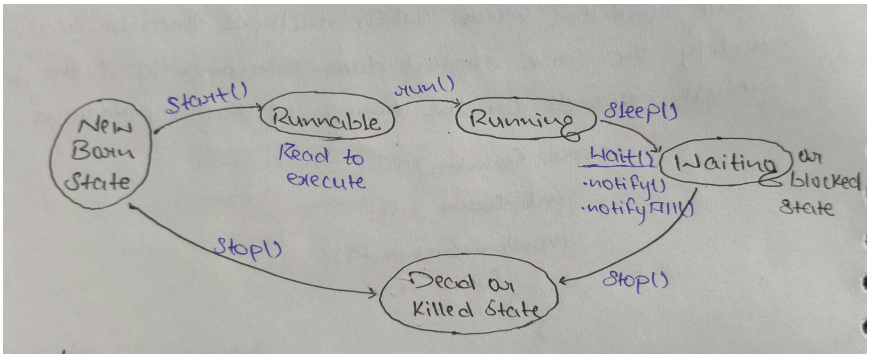
Preventing a race condition

```
class Counter {
    int count = 0;

    synchronized void increment() {
        count++;
    }
}
```

Only one thread at a time can execute the synchronized method for the same object.

Thread states



1. New/Born state

The thread object is created, but `start()` has not been called.

```
Thread t = new Thread();
```

2. Runnable state

After `t.start()`, the thread becomes runnable/ready to execute.

3. Running state

When the thread scheduler selects the thread, its `run()` method starts executing.

Java does not have a separate `RUNNING` value in `Thread.State`; a running thread is considered `RUNNABLE`.

4. Waiting state

A running thread may temporarily stop executing and wait for some time.

```
Thread.sleep(1000);
```

Other causes include `wait()` and `join()`.

5. Dead/terminated state

- When the `run()` method finishes, the thread has completed execution.
- After `run()` completes, the thread becomes terminated.
- The same thread cannot be restarted; create a new thread object.

Collections API, Lists, Sets, Maps, and Sorting

Collection API

Collection API is a framework of interfaces and classes used to store and manipulate groups of objects.

```
Collection
  |- List
  |    |- ArrayList
  |    |- LinkedList
  |    `- Vector
  |- Set
  |    |- HashSet
```

```
  |    |- LinkedHashSet
  |    `- TreeSet
  `- Queue
       |- PriorityQueue
       `- Deque

Map (separate from Collection)
  |- HashMap
  |- LinkedHashMap
  `- TreeMap
```

Why Collection API?

Arrays have limitations:

- Fixed size.
- Limited built-in operations.
- Difficult insertion/deletion.
- No ready-made searching/sorting utilities.

Collections provide:

- Dynamic size.
- Searching.
- Sorting.
- Adding/removing elements.
- Duplicate handling.
- Key-value storage.

ArrayList in Java

`ArrayList` is a resizable-array implementation of the `List` interface.

```java
ArrayList<String> students = new ArrayList<>();
students.add("Rahul");
```

```
students.add("Aman");
System.out.println(students);
```

Important properties

- Maintains insertion order.
- Allows duplicates.
- Allows `null`.
- Dynamic/resizable.
- Allows index-based access.
- Stores objects, not primitive types directly; use `Integer`, not `int`.

Set in Java

A `Set` is a collection that does not allow duplicate elements.

Main Set implementations

- `HashSet` - no guaranteed order.
- `LinkedHashSet` - maintains insertion order.
- `TreeSet` - sorted order.

```
Set<Integer> numbers = new HashSet<>();
numbers.add(10);
numbers.add(20);
numbers.add(10);
System.out.println(numbers); // 10 and 20 only
```

Map in Java

A `Map` stores data in key-value pairs.

```java
Map<Integer, String> students = new HashMap<>();
students.put(101, "Rahul");
students.put(102, "Aman");
System.out.println(students.get(101));
```

Important properties

- Keys are unique.
- Values can be duplicated.
- Each key maps to one value.
- Data is accessed using keys.

Main Map implementations

- `HashMap` - fast lookup; no guaranteed order.
- `LinkedHashMap` - maintains insertion order.
- `TreeMap` - sorted by keys.
- `Hashtable` - legacy synchronized map.

Comparable vs. Comparator

Both are used for sorting objects.

`Comparable`

Use `Comparable` when the class defines its natural/default ordering.

```java
class Student implements Comparable<Student> {
    int age;

    Student(int age) {
        this.age = age;
    }
```

```
    @Override
    public int compareTo(Student s) {
        return this.age - s.age;
    }
}

Collections.sort(students);
```

- Package: `java.lang`.
- Method: `compareTo()`.
- Sorting logic is inside the class.
- Defines natural ordering.

`Comparator`

Use `Comparator` when different/custom sorting rules are needed.

```
Comparator<Student> byAge = (s1, s2) -> s1.age - s2.age;
```

- Package: `java.util`.
- Method: `compare()`.
- Sorting logic can be outside the class.
- Allows multiple sorting strategies.

Stream API

Why Stream API?

Before streams, code often used a loop:

```
for (Student s : students) {
    if (s.getAge() > 18) {
```

```
            System.out.println(s.getName());
        }
    }
```

Streams express the same operation more declaratively.

```
students.stream()
        .filter(s -> s.getAge() > 18)
        .forEach(s -> System.out.println(s.getName()));
```

Why use Stream API?

- Cleaner collection processing.
- More readable code.
- Less boilerplate.
- Easier to chain operations.
- Convenient for filtering, mapping, sorting, grouping, and more.
- Supports parallel processing when appropriate.

Stream does not store data:

```
Collection -> stores data
Stream -> processes data
```

forEach() in Java

`forEach()` performs an action for each element.

```
List<String> students = Arrays.asList("Rahul", "Aman", "Rav
i");
students.forEach(name -> System.out.println(name));
```

It is commonly used with lambda expressions.

Stream API in Java

- Stream API was introduced in Java 8.
- A stream represents a sequence of elements on which operations can be performed.

```
List<Integer> numbers = Arrays.asList(10, 15, 20, 25, 30);

numbers.stream()                    // source
       .filter(n -> n > 20)         // intermediate operation
       .forEach(n -> System.out.println(n)); // terminal oper
ation
```

Stream pipeline

```
Source -> Intermediate operation -> Terminal operation
```

Important stream operations

1. `filter()` - selects elements based on a condition.
2. `map()` - transforms each element.
3. `sorted()` - sorts elements.
4. `distinct()` - removes duplicates.
5. `limit()` - takes the first `N` elements.
6. `count()` - counts elements.
7. `collect()` - collects stream results into a collection.
8. `reduce()` - combines elements into one result.

---

1. Generics

Why Generics?

- Provide compile-time type safety.
- Prevent wrong data types from entering collections.
- Remove the need for explicit casting.
- Improve code reusability.

```
List<String> names = new ArrayList<>();
names.add("Asha");
String name = names.get(0);
```

Generic Class

```
class Box<T> {
    T value;

    Box(T value) {
        this.value = value;
    }
}
```

Generic Method

```
static <T> void print(T value) {
    System.out.println(value);
}
```

Bounds and Wildcards

- `<T extends Number>`: T must be `Number` or its subclass.
- `?`: unknown type.
- `? extends T`: use when reading/producing values.
- `? super T`: use when adding/consuming values.

- Rule: **PECS - Producer Extends, Consumer Super.**

Type Erasure

Java removes most generic type information at runtime. Generics mainly provide compile-time safety.

---

2. Collection Internals and Comparison

ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Internal structure	Dynamic array	Doubly linked nodes
Index access	Fast - `O(1)`	Slow - `O(n)`
Insert/remove middle	Usually `O(n)`	Better when node position is known
Default choice	Yes	Only for specific insertion/deletion use cases

HashMap Internal Working

- Stores data as key-value pairs.
- Uses `hashCode()` to find the bucket.
- Uses `equals()` to find the exact key.
- Average `put()` and `get()` time complexity: `O(1)`.
- Allows one `null` key and multiple `null` values.
- Is not thread-safe.

```
Map<String, Integer> marks = new HashMap<>();
marks.put("Riya", 92);

System.out.println(marks.get("Riya"));
```

Set Types

Collection	Duplicates	Order	Complexity
HashSet	Not allowed	No guaranteed order	`O(1)` average
LinkedHashSet	Not allowed	Insertion order	`O(1)` average
TreeSet	Not allowed	Sorted order	`O(log n)`

Map Types

Collection	Order
HashMap	No guaranteed order
LinkedHashMap	Insertion order
TreeMap	Sorted by key

Iterator and Fail-Fast

- Do not modify a collection directly inside a for-each loop.
- It can cause `ConcurrentModificationException`.
- Use `Iterator.remove()` instead.

```
Iterator<String> it = list.iterator();

while (it.hasNext()) {
    if (it.next().equals("old")) {
        it.remove();
    }
}
```

3. Strings, Equality, and Object Contract

String Pool

String literals are stored and reused in the String pool.

```java
String a = "Java";
String b = "Java";
String c = new String("Java");

System.out.println(a == b);       // true
System.out.println(a == c);       // false
System.out.println(a.equals(c));  // true
```

`==` vs `equals()`

- `==` compares object references.
- `equals()` compares content.
- Use `equals()` for String content comparison.

Why is String Immutable?

- Supports the String pool.
- Improves security.
- Makes Strings thread-safe.
- Allows cached hash codes.

`equals()` and `hashCode()` Contract

- If `a.equals(b)` is `true`, then `a.hashCode() == b.hashCode()` must also be true.
- Override both methods together.
- Important for `HashMap`, `HashSet`, and other hash-based collections.

---

4. JVM Memory and Garbage Collection

JVM Components

- **Class Loader**: loads `.class` files.

- **Method Area / Metaspace**: stores class metadata.
- **Heap**: stores objects and arrays.
- **Stack**: stores method calls and local variables.
- **JIT Compiler**: improves execution speed by compiling frequently used bytecode.

Garbage Collection

- Garbage Collector removes objects that are no longer reachable.
- `System.gc()` only requests garbage collection; it does not guarantee it.
- Java can still have memory leaks when unused objects remain referenced.

Java is Pass-by-Value

- Primitives: a copy of the value is passed.
- Objects: a copy of the reference is passed.
- A method can modify an object, but cannot replace the caller's reference.

```
static void changeName(Student s) {
    s.name = "New";        // visible outside method
    s = new Student();     // not visible outside method
}
```

---

5. Modern Concurrency Basics

`volatile`

- Ensures visibility of the latest value between threads.
- Does not make operations like `count++` atomic.

```
volatile boolean running = true;
```

`wait()`, `notify()`, and `notifyAll()`

- Used for communication between threads.
- Must be called inside synchronized code.
- `wait()` releases the lock while waiting.
- `sleep()` does not release the lock.

ExecutorService

Use thread pools instead of creating a new thread for every task.

```
ExecutorService pool = Executors.newFixedThreadPool(2);

Future<Integer> result = pool.submit(() -> 2 + 3);

System.out.println(result.get());

pool.shutdown();
```

- `submit()` returns a `Future`.
- `shutdown()` stops accepting new tasks.

---

6. File Handling and NIO

`Path` and `Files`

```
Path path = Path.of("notes.txt");

Files.writeString(path, "Core Java");

String text = Files.readString(path);
```

- `Path` represents a file location.
- `Files` provides methods for reading, writing, copying, moving, and deleting files.

- Modern Java code should prefer `Path` and `Files`.

Try-with-Resources

Resources close automatically after use.

```
try (BufferedReader br = Files.newBufferedReader(path)) {
    System.out.println(br.readLine());
}
```

- Works with classes implementing `AutoCloseable`.
- Prevents resource leaks.

---

7. Serialization

Meaning

Serialization converts an object into bytes for storage or transfer.

```
class User implements Serializable {
    private static final long serialVersionUID = 1L;

    String name;
    transient String password;
}
```

Important Points

- Class must implement `Serializable`.
- `transient` fields are not serialized.
- `serialVersionUID` supports version compatibility.
- Never deserialize untrusted data without validation.

---

8. JDBC Basics

JDBC Flow

1. Create database connection.
2. Create `PreparedStatement`.
3. Set parameter values.
4. Execute query or update.
5. Read `ResultSet` if required.
6. Close resources.

```
String sql = "SELECT name FROM student WHERE id = ?";

try (Connection con = DriverManager.getConnection(url, use
r, password);
     PreparedStatement ps = con.prepareStatement(sql)) {

    ps.setInt(1, 101);

    ResultSet rs = ps.executeQuery();
}
```

`executeQuery()` vs `executeUpdate()`

- `executeQuery()`:
  - Used for `SELECT`.
  - Returns `ResultSet`.
- `executeUpdate()`:
  - Used for `INSERT`, `UPDATE`, and `DELETE`.
  - Returns affected row count.

Why PreparedStatement?

- Prevents SQL injection.
- Separates SQL from user input.
- Easier to read and reuse.

---

9. Modern Date and Time API

```
LocalDate today = LocalDate.now();

LocalDate nextWeek = today.plusDays(7);

System.out.println(nextWeek);
```

Important Classes

- `LocalDate`: date only.
- `LocalTime`: time only.
- `LocalDateTime`: date and time without timezone.
- `ZonedDateTime`: date and time with timezone.
- `Duration`: time-based amount.
- `Period`: date-based amount.

> Prefer `java.time` over old `Date` and `Calendar` classes.